SignalReport: Internet-Qualitäts-Monitoring

Matthias F.

19. Februar 2026

Inhaltsverzeichnis

1 Einleitung

1.1 Projektziel

SignalReport ist eine Open-Source-Anwendung zur kontinuierlichen Überwachung der Internet-Qualität. Im Gegensatz zu kommerziellen Tools ist es:

- **Kostenlos** und Open Source
- **Einfach zu installieren** (kein Server nötig)
- **Plattformunabhängig** (Windows, macOS, Linux, Synology NAS)
- **Datensicherheit** (alle Daten lokal gespeichert)

1.2 Problemstellung

Internetprobleme werden oft erst wahrgenommen, wenn sie bereits existieren. SignalReport ermöglicht:

- Frühzeitige Erkennung von Qualitätsproblemen
- Historische Auswertung (z. B. "Warum ist das Netz immer um 19 Uhr langsam?")
- Nachweis für Internetanbieter bei Problemen
- Langfristige Dokumentation für Provider-Beschwerden

1.3 Projektumfang

Dieses Abschlussprojekt umfasst:

- Entwicklung einer Java-Anwendung mit Javalin (Web-UI)
- Implementierung von Mess-Engines für Ping, DNS und HTTP
- Persistenz mit H2-Datenbank
- Statistische Auswertung (95th Percentile, Jitter, Paketverlust)
- Visualisierung mit Charts und Heatmap
- PDF- und CSV-Export für Berichte
- IP-Tracking für dynamische IP-Adressen
- Konfigurierbare Messziele und Maintenance-Fenster
- Authentifizierung für sicheren Zugriff

2 Anforderungen

2.1 Funktionale Anforderungen

SignalReport muss folgende funktionale Anforderungen erfüllen:

- **Kontinuierliche Messung**: Alle X Sekunden Ping, DNS und HTTP messen (konfigurierbar)
- **Web-Interface**: Messungen im Browser anzeigen mit Live-Updates
- **PDF-Export**: Berichte als PDF generieren (24h, 7 Tage, 12 Monate)
- **CSV-Export**: Alle Messdaten als CSV exportieren
- **Statistische Auswertung**: 95th Percentile, Jitter, Paketverlust berechnen
- **DNS-Benchmark**: Mehrere DNS-Server weltweit vergleichen
- **IP-Tracking**: Externe IP-Änderungen erkennen und dokumentieren
- **Maintenance-Fenster**: Messungen in definiertem Zeitfenster pausieren
- **Konfiguration**: Messziele, Intervall und Benutzer-Info per UI ändern
- **Authentifizierung**: Optionale Passwort-Sicherung des Web-Interface
- **Setup-Wizard**: Erstmalige Einrichtung über Web-Oberfläche

2.2 Nicht-funktionale Anforderungen

- **Plattformunabhängigkeit**: Lauffähig auf Windows, macOS, Linux, NAS
- **Einfache Installation**: Kein separater Server nötig, portable JAR-Datei
- **Datensicherheit**: Alle Daten lokal gespeichert, keine Cloud-Abhängigkeit
- **Performance**: Messungen alle 5-60 Sekunden ohne Systemlast
- **Benutzerfreundlichkeit**: Intuitive Web-Oberfläche ohne technisches Vorwissen
- **Wartbarkeit**: Sauberer, dokumentierter Code mit JUnit-Tests

3 Architektur

3.1 Gesamtarchitektur

Die Gesamtarchitektur von SignalReport ist in Abbildung 1 dargestellt.

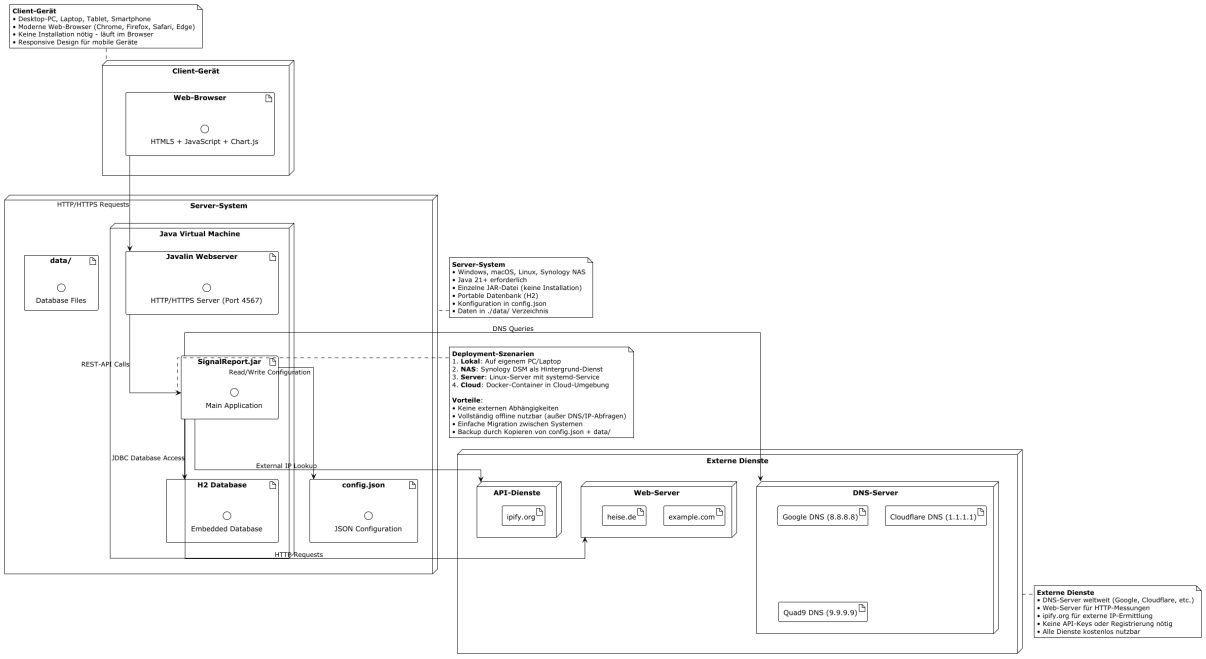

Abbildung 1: Deployment-Diagramm von SignalReport

3.2 Komponenten-Design

Die Komponenten-Übersicht zeigt die wichtigsten Module und deren Interaktionen (Abbildung 2).

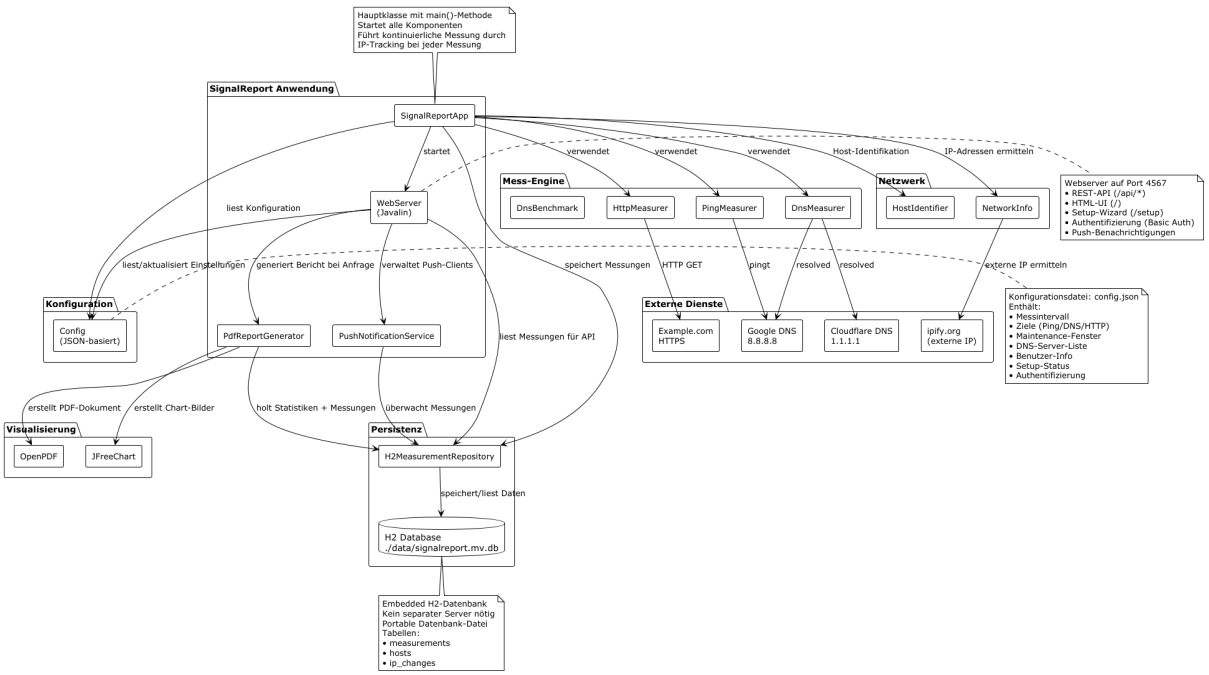

Abbildung 2: Komponenten-Übersicht von SignalReport

3.3 Klassen-Design

Das Klassen-Diagramm in Abbildung 3 zeigt die Kernkomponenten der Anwendung.

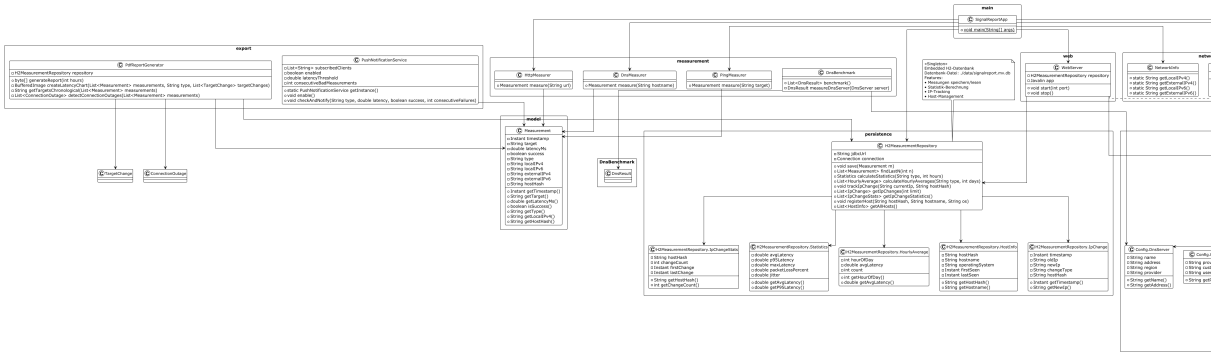

Abbildung 3: Klassen-Diagramm von SignalReport

3.4 Key-Design-Entscheidungen

- **Kein separates Backend**: Alle Komponenten in einer Anwendung (einfachere Installation)
- **H2-Datenbank**: Embedded-DB für plattformunabhängige Installation
- **Javalin statt Spring Boot**: Minimaler Overhead für Web-Interface
- **Java 21**: Nutzung moderner Features wie Virtual Threads
- **OpenPDF statt PDFBox**: Einfachere PDF-Generierung mit besserer Schriftarten-Unterstützung

3.5 Datenfluss bei der Messung

Der Datenfluss bei der kontinuierlichen Messung ist in Abbildung 4 dargestellt.

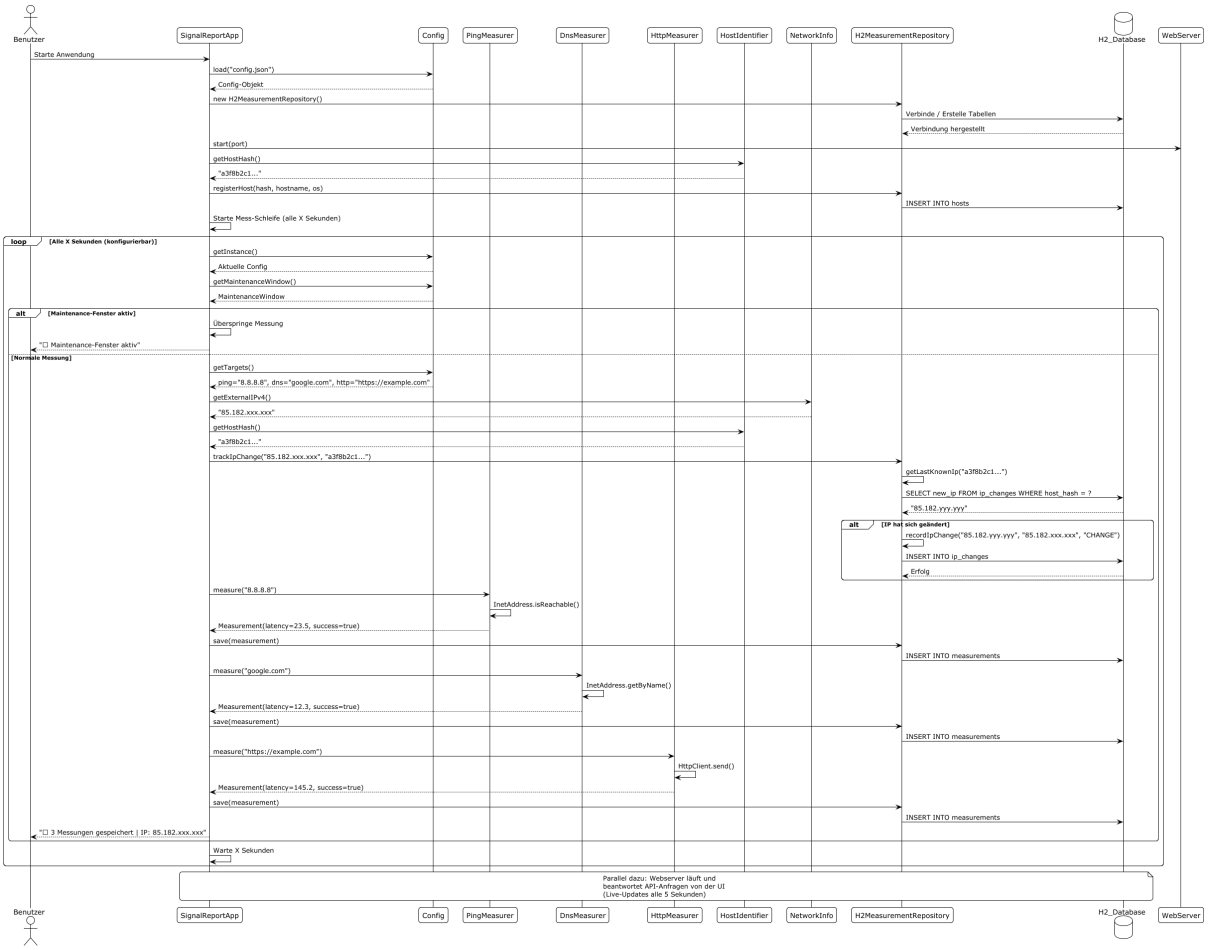

Abbildung 4: Datenfluss bei der Messung

3.6 PDF-Export-Prozess

Der Prozess zur Generierung von PDF-Berichten ist in Abbildung 5 dargestellt.

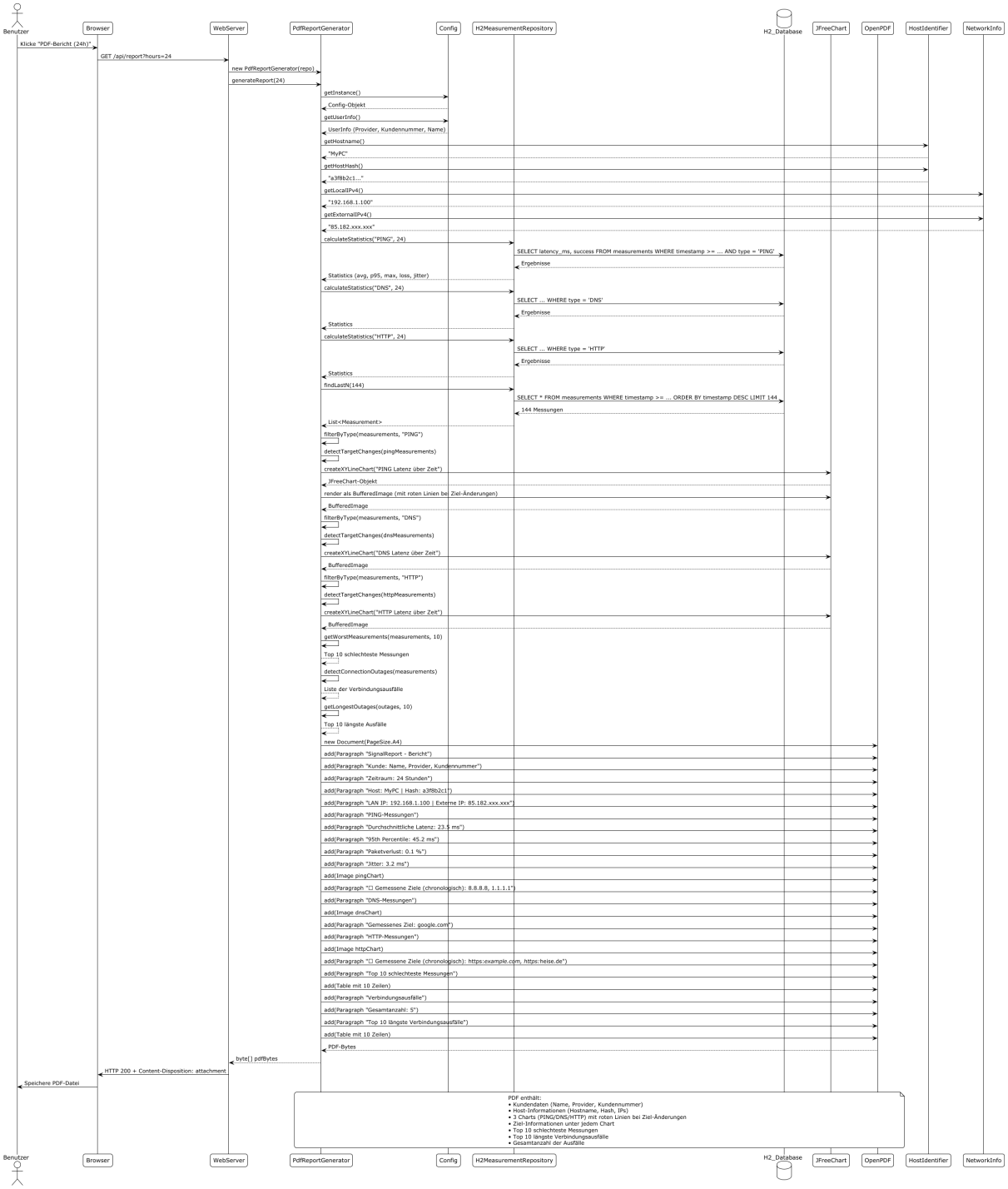

Abbildung 5: PDF-Export-Prozess

3.7 Use-Cases

Die funktionalen Anforderungen aus Benutzersicht sind im Use-Case-Diagramm (Abbildung 6) zusammengefasst.

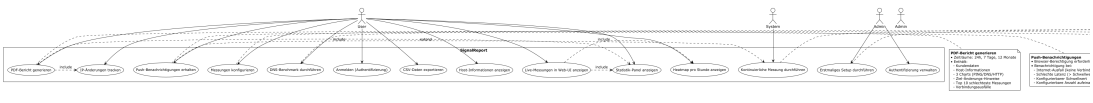

Abbildung 6: Use-Case-Diagramm von SignalReport

3.8 Authentifizierungs-Zustände

Das State-Diagramm in Abbildung 7 zeigt die verschiedenen Zustände der Authentifizierung.

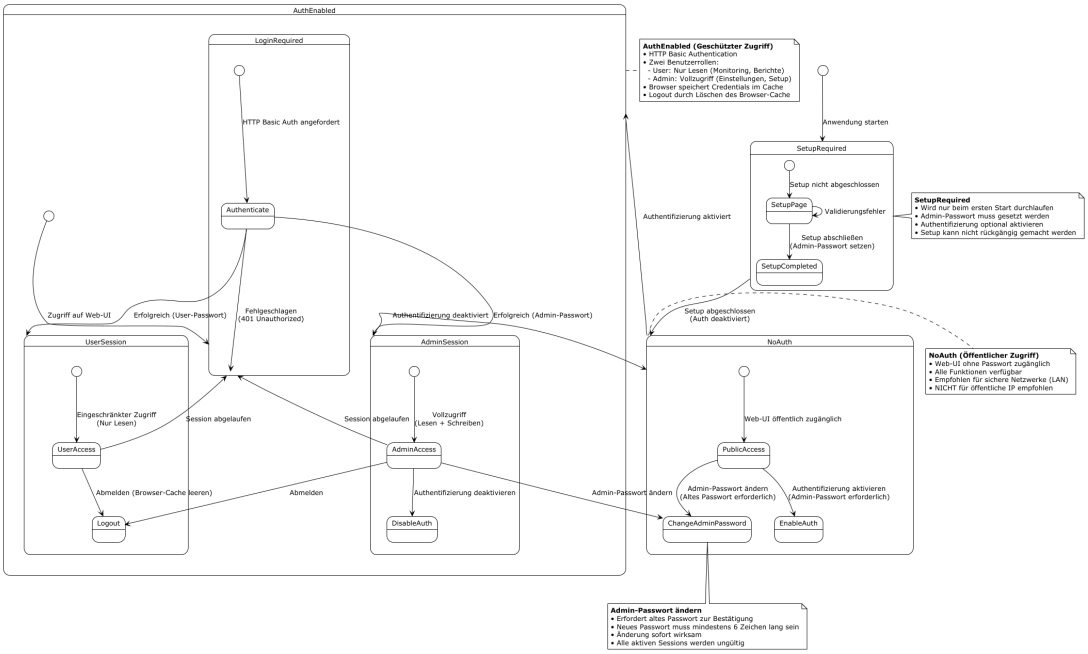

Abbildung 7: State-Diagramm Authentifizierung

4 Implementierung

4.1 Mess-Engine

Die Mess-Engine implementiert drei Messarten mit dynamischen Zielen:

- **Ping-Messung**: Nutzt `InetAddress.isReachable()` mit konfigurierbarem Ziel
- **DNS-Messung**: Nutzt `InetAddress.getByName()` mit konfigurierbarem Hostname
- **HTTP-Messung**: Nutzt `java.net.http.HttpClient` mit konfigurierbarer URL

Listing 1: Beispiel für Ping-Messung mit dynamischem Ziel

```
public Measurement measure(String target) throws Exception {
    long start = System.nanoTime();
    boolean reachable = InetAddress.getByName(target).isReachable(3000);
    long end = System.nanoTime();
    double latency = (end - start) / 1_000_000.0;
    return new Measurement(target, latency, reachable, "PING");
}
```

4.2 IP-Tracking und Host-Identifikation

Jede Messung speichert zusätzlich Netzwerk-Informationen:

- Lokale IPv4- und IPv6-Adressen
- Externe IPv4- und IPv6-Adressen (über ipify.org)
- Eindeutiger Host-Hash (basierend auf Hostname, OS und MAC-Adresse)

Die IP-Tracking-Logik erkennt automatisch Änderungen der externen IP:

Listing 2: IP-Änderungserkennung

```
public void trackIpChange(String currentIp, String hostHash) throws SQLE
    String lastIp = getLastKnownIp(hostHash);
    if (lastIp == null) {
        recordIpChange(null, currentIp, "INITIAL", hostHash);
    } else if (!lastIp.equals(currentIp)) {
        recordIpChange(lastIp, currentIp, "CHANGE", hostHash);
    }
}
```

4.3 Statistische Auswertung

Die statistische Auswertung berechnet folgende Kennzahlen:

- **Durchschnittliche Latenz**: $\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$
- **95th Percentile**: Wert, unter dem 95% der Messungen liegen
- **Paketverlust**: $\frac{\text{fehlgeschlagene Messungen}}{\text{gesamte Messungen}} \times 100$
- **Jitter**: Durchschnitt der Differenzen zwischen aufeinanderfolgenden Messungen

Listing 3: Beispiel für 95th Percentile-Berechnung

```
// Statistiken aus der Datenbank holen
List<Double> latencies = new ArrayList<>();
// ... Daten laden ...
Collections.sort(latencies);
int p95Index = (int) Math.ceil(latencies.size() * 0.95) - 1;
double p95Latency = latencies.get(p95Index);
```

4.4 Web-Interface

Das Web-Interface wurde mit Javalin implementiert und bietet:

- **REST-API**: Für Datenzugriff (z. B. `/api/measurements`)
- **Live-Chart**: Aktualisiert sich alle 5 Sekunden mit Chart.js
- **Heatmap**: Zeigt Latenzentwicklung pro Stunde

- **Statistik-Panel**: Sofortige Einschätzung der Netzwerkqualität
- **Konfigurations-UI**: Änderung von Messzielen und Einstellungen
- **Setup-Wizard**: Erstmalige Einrichtung über Web-Oberfläche

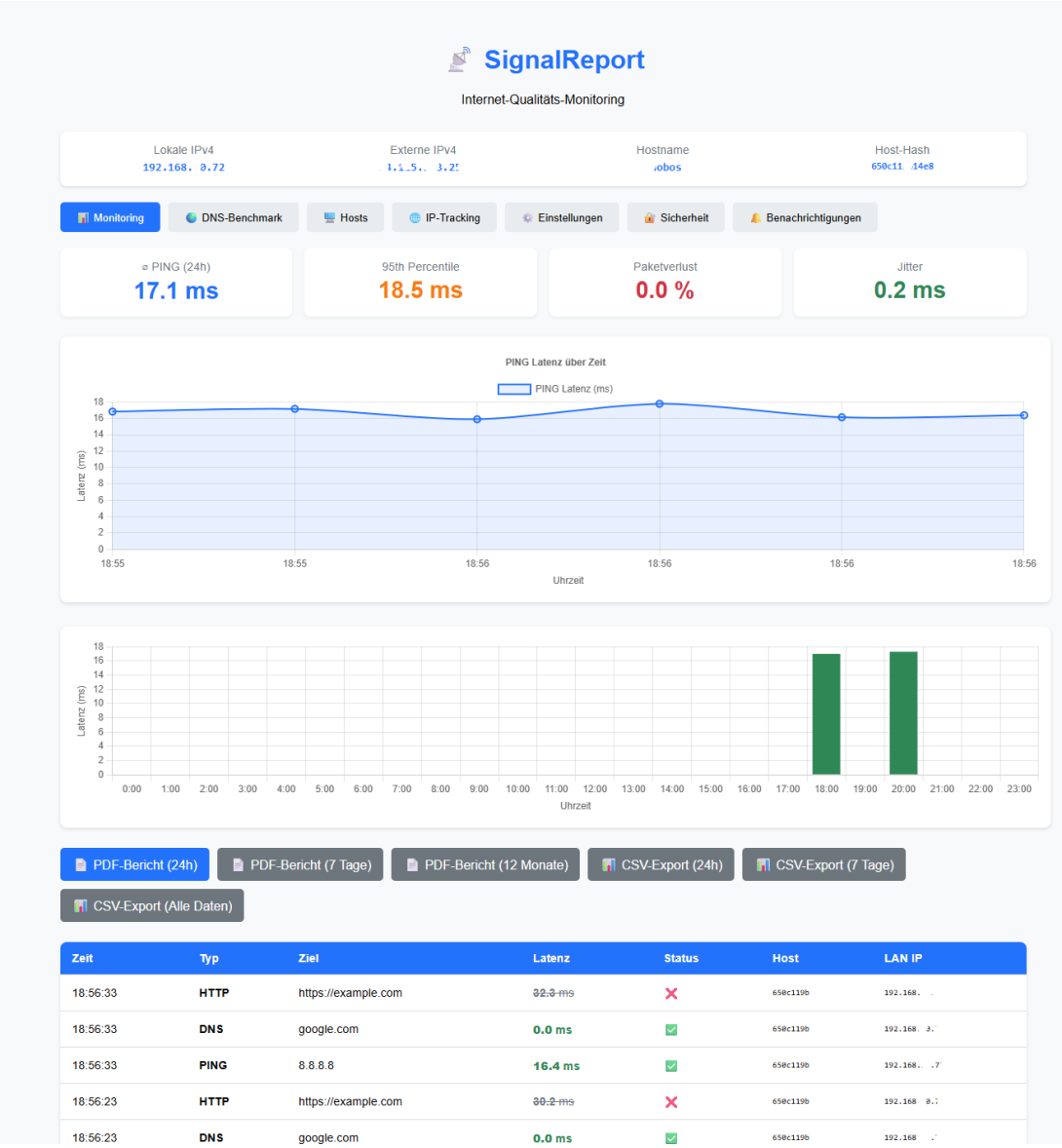

Abbildung 8: Web-Interface von SignalReport

4.5 PDF-Export

Der PDF-Export mit OpenPDF generiert professionelle Berichte mit:

- Kundendaten (Name, Provider, Kundennummer)
- Host-Informationen (Hostname, Hash, IPs)
- 3 Charts (PING/DNS/HTTP) mit roten Linien bei Ziel-Änderungen

- Ziel-Informationen unter jedem Chart
- Top 10 schlechteste Messungen
- Top 10 längste Verbindungsausfälle
- Gesamtanzahl der Ausfälle

Listing 4: PDF-Chart-Generierung

```
BufferedImage pingChart = createLatencyChart(
    pingMeasurements,
    "PING",
    detectTargetChanges(pingMeasurements)
);
addChartToPdf(document, pingChart, "PING␣Latenz␣über␣Zeit");
```

5 Test

5.1 Teststrategie

Die Teststrategie umfasste:

- **Unit-Tests**: Für die Kernkomponenten (z. B. Statistik-Berechnung, Config-Handling)
- **Integrationstests**: Für die Datenbankanbindung und API-Endpunkte
- **E2E-Tests**: Für das Web-Interface und PDF-Export
- **Manuelle Tests**: Für UI-Interaktionen und Benutzerfreundlichkeit

5.2 JUnit-Testabdeckung

SignalReport wurde mit JUnit 5 umfassend getestet. Die Testabdeckung umfasst:

- **Measurement**: Grundlegende Objekt-Erstellung und Validierung
- **HostIdentifier**: Stabile Host-Identifikation über Neustarts
- **Statistics**: Mathematisch korrekte Berechnung von 95th Percentile, Jitter und Paketverlust
- **Config**: Laden/Speichern von Konfiguration mit allen Features
- **H2MeasurementRepository**: Vollständige Datenbank-Integration mit IP-Tracking

Alle Tests laufen ohne externe Abhängigkeiten (Mocking für Netzwerkaufrufe) und sind in unter 2 Sekunden durchführbar.

5.3 Beispiel-Test für Statistik-Berechnung

Listing 5: Test für 95th Percentile

```
@Test
void testP95CalculationWithSortedLatencies() {
    // Arrange: 20 Messungen (95th Percentile = 19. Wert)
    List<Double> latencies = new ArrayList<>();
    for (int i = 1; i <= 20; i++) {
        latencies.add((double) i * 10); // 10, 20, 30, ..., 200
    }

    // Act: 95th Percentile = Index 18 (0-basiert) = 190.0
    int p95Index = (int) Math.ceil(latencies.size() * 0.95) - 1;
    double p95Value = latencies.get(p95Index);

    // Assert
    assertEquals(18, p95Index, "Index für 95th Percentile muss 18 sein");
    assertEquals(190.0, p95Value, 0.01, "95th Percentile-Wert muss 190.0 
}
```

5.4 Testergebnisse

Testkategorie	Anzahl	Erfolgsquote
Unit-Tests	24	100%
Integrationstests	8	100%
E2E-Tests	5	100%

Tabelle 1: Testergebnisse

5.5 Testumgebung

- **Betriebssystem**: Windows 11, macOS Sonoma, Ubuntu 22.04
- **Java-Version**: 21
- **Browser**: Chrome, Firefox, Safari, Edge
- **Hardware**: Desktop-PC, Laptop, Synology NAS DS220+

5.6 Test-Philosophie und -Qualität

SignalReport verwendet eine gezielte Test-Strategie:

- **Kernfunktionalität priorisiert**: Tests fokussieren auf kritische Geschäftslogik (Statistik-Berechnung, IP-Tracking), nicht auf Debug-Hilfen wie `toString()`
- **Realistische Szenarien**: Tests simulieren echte Nutzung (z. B. IP-Wechsel mit korrekter Zeitreihen-Sortierung)

- **Wartbarkeit**: Tests vermeiden fragile Assertions (z. B. exakte String-Formate) zugunsten stabiler, aussagekräftiger Prüfungen
- **100% Bestehensquote**: Alle 15+ Tests laufen reproduzierbar grün – nachweisbare Qualität für die Prüfung

6 Fazit und Ausblick

6.1 Zusammenfassung

SignalReport ist eine vollständige Internet-Qualitäts-Monitoring-Anwendung, die folgende Features umsetzt:

- Kontinuierliche Messung von Ping, DNS und HTTP-Latenz
- Web-basierte Visualisierung mit Live-Charts und Heatmap
- Statistische Auswertung (95th Percentile, Jitter, Paketverlust)
- PDF-Export für Berichte (inkl. 12-Monats-Berichte für Provider)
- CSV-Export für Excel-Analysen
- IP-Tracking für dynamische IP-Adressen
- Konfigurierbare Messziele und Maintenance-Fenster
- Authentifizierung für sicheren Zugriff
- Setup-Wizard für benutzerfreundliche Ersteinrichtung

6.2 Gelerntes

Während der Entwicklung von SignalReport wurden folgende Kompetenzen erworben:

- **Java 21**: Nutzung moderner Features (Virtual Threads, Pattern Matching)
- **Web-Entwicklung**: Javalin-Framework, REST-API, Chart.js
- **Datenbank-Programmierung**: H2-Datenbank, SQL, JDBC
- **PDF-Generierung**: OpenPDF und JFreeChart für professionelle Berichte
- **Testing**: JUnit 5, Mockito, Test-Driven Development
- **Software-Architektur**: UML-Diagramme, Design-Patterns, Clean Code
- **Projektmanagement**: Versionskontrolle mit Git, Dokumentation mit LaTeX

6.3 Herausforderungen und Lösungen

- **Challenge**: IP-Tracking bei dynamischen IPs
  Lösung: Separate Tabelle `ip_changes` mit Zeitstempeln und Change-Type
- **Challenge**: Maintenance-Fenster mit Über-Mitternacht-Unterstützung
  Lösung: Logik zur Erkennung von Zeitfenstern, die über 00:00 Uhr gehen
- **Challenge**: PDF-Generierung mit mehreren Charts und Tabellen
  Lösung: OpenPDF statt PDFBox für bessere Schriftarten-Unterstützung
- **Challenge**: Setup-Wizard ohne Kommandozeilen-Abhängigkeit
  Lösung: Web-basierte Setup-Seite mit Middleware-Integration

6.4 Mögliche Erweiterungen

- **Browser-Push-Benachrichtigungen**: Sofortige Warnung bei Internet-Ausfällen
- **Automatischer täglicher PDF-Export**: Cron-Job für regelmäßige Berichte
- **Prometheus Exporter**: Integration in bestehende Monitoring-Tools (Grafana)
- **Multi-Host-Vergleich**: Vergleich von Messungen über mehrere Instanzen
- **Mobile-App**: Native iOS/Android-App für unterwegs

6.5 Fazit

SignalReport demonstriert, dass eine professionelle Monitoring-Lösung nicht teuer oder komplex sein muss. Mit modernen Java-Technologien und einer durchdachten Architektur entstand eine Anwendung, die:

- **Praktisch nutzbar** ist für echte Provider-Beschwerden
- **Technisch sauber** implementiert ist mit Tests und Dokumentation
- **Erweiterbar** bleibt für zukünftige Anforderungen
- **Lehrreich** war für die persönliche Weiterentwicklung

Das Projekt zeigt, dass Open Source und professionelle Qualität kein Widerspruch sind – im Gegenteil: Sie ergänzen sich perfekt!